Scientific Reports

https://doi.org/10.1038/s41598-025-32753-w

Article in Press

Computing power network dynamic resource scheduling integrating time series mixing dynamic state estimation and hierarchical reinforcement learning

Received: 24 September 2025

Accepted: 12 December 2025

Published online: 04 January 2026

Cite this article as: Liu H., Zhang S., Li L. *et al.* Computing power network dynamic resource scheduling integrating time series mixing dynamic state estimation and hierarchical reinforcement learning. *Sci Rep* (2026). https://doi.org/10.1038/s41598-025-32753-w

Hong Liu, Shulei Zhang, Li Li, Tingting Sun, Wei Xue, Xiaoxia Yao & Yong Xu

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

ARTICLE IN PRESS

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

Computing Power Network Dynamic Resource Scheduling Integrating Time Series Mixing Dynamic State Estimation and Hierarchical Reinforcement Learning

Hong Liu[1], Shulei Zhang[1,*], Li Li[1], Tingting Sun[1], Wei Xue[1], Xiaoxia Yao[1], and Yong Xu[1]

[1]China Mobile Information Technology Co., Ltd., Beijing, 100033
[*]Shulei Zhang(Corresponding author): zshulll@163.com

ABSTRACT

With the evolution of cloud computing towards a multi-cloud architecture, cross-cloud resource scheduling faces challenges such as heterogeneous environment adaptation and slow dynamic load response. How to improve resource utilization while ensuring service quality has become a core challenge in the field of cloud management. To address this need, we propose the TSL-HRL intelligent scheduling framework, which integrates time-series feature modeling and hierarchical reinforcement learning. The framework utilizes a time-series mixing module to deeply mine the periodic fluctuations and burst demand features of computing, storage, and network resources. It integrates a dynamic state estimation module with Kalman filtering to capture real-time changes in resource supply and demand. Additionally, it constructs a high-level planning - low-level response hierarchical reinforcement learning architecture: the high-level Q-learning algorithm formulates a global long-term resource allocation strategy to ensure optimal overall scheduling, while the low-level A2C algorithm adjusts the execution plan based on real-time network fluctuations and node load, enabling fast adaptation to dynamic changes, forming a macro-micro collaborative decision mechanism. In experiments on the Multi-Cloud Service Composition Dataset and Google 2019 Cluster dynamic node scenarios, TSL-HRL effectively balanced resource utilization efficiency and scheduling real-time performance with its three-level architecture design of time-series feature extraction - dynamic state perception - hierarchical strategy optimization. The study shows that TSL-HRL provides a systematic solution for resource management in multi-cloud environments. Future research will focus on lightweight extensions for edge-cloud collaborative scenarios, multi-objective energy consumption optimization frameworks, and meta-learning-driven rapid adaptation technologies, promoting the application and generalization of intelligent resource scheduling technologies in real-world complex scenarios.

keywords: Computing Power Network; Dynamic Resource Scheduling; Hierarchical Reinforcement Learning; Time-Series Feature Modeling; TSL-HRL Model; Dynamic State Estimation

Introduction

Research Background

In the era of rapid digital economy development, the integration of cutting-edge technologies such as cloud computing, edge computing, and artificial intelligence is accelerating, driving computing power networks into a new stage of development. With the widespread adoption of 5G, the Internet of Things (IoT), and other technologies, emerging business scenarios such as online education, e-commerce promotions, industrial IoT, and intelligent transportation are continuously emerging[1–3]. These businesses feature strong burstiness and high real-time demand for traffic, making traditional static and single resource reservation models inadequate to meet the dynamic changes in computing power networks. Traditional resource reservation

ARTICLE IN PRESS

models mostly rely on historical average data or simple heuristics. In industrial IoT scenarios, such as car manufacturing factories, production line equipment needs real-time parameter collection for quality monitoring and fault warning[4]. The resource requirements for data processing tasks vary significantly at different stages of production. Traditional methods often fail to match resources precisely, leading to resource shortages that halt production lines, or underutilization of resources causing waste and increased costs[5]. The traditional fixed-time or uniform-quota resource allocation model fails to reflect the actual resource usage and business priority differences. For example, in the field of medical image processing, emergency consultations involving image data can impact patient health, requiring timely resource allocation, yet resources for these tasks are allocated similarly to regular image storage[6,7]. This fails to incentivize service providers to prioritize critical tasks and does not guide users in utilizing resources efficiently.

Literature Review

Advances in Time Series Prediction and Resource Management

In time series prediction, traditional ARIMA relies on data stationarity and linear correlation modeling but fails to capture nonlinear fluctuations in computing power network resource usage, driven by business activities and user behaviors[8,9]. LSTM networks address long-term dependencies effectively but lack multimodal data fusion capability, hindering the linkage of device status, network traffic, and user requests and thus impacting prediction accuracy[10–12]. Transformer models excel at long-sequence dependency capture. Recent hybrid frameworks such as VMD-Transformer optimize load prediction by decomposing non-stationary sequences and refining features, mitigating Transformer's noise sensitivity[13]. However, Transformers suffer from high computational complexity incompatible with edge computing, focusing only on single-modal load data and lacking adaptation to multi-cloud heterogeneity or dynamic changes[14,15].TSMixer aligns with PatchTST and TFT in efficient resource-related sequence forecasting: PatchTST uses patch-based self-attention[16], TFT integrates gating mechanisms[17], while TSMixer adopts an attention-free mixing design to balance computational efficiency and feature capture, meeting real-time requirements of computing power networks. Hybrid CNN-LSTM models capture spatiotemporal features but require extensive parameter tuning, limiting scalability in practical applications[18,19].

For resource management, early rule-driven algorithms (FIFO, SJF) ignore task requirements and network status, causing resource imbalance that prolongs critical task waiting times and reduces processing efficiency and resource utilization[20–22]. Linear and integer programming provide static optimal solutions but face dynamic network challenges, as frequent adjustments increase computational complexity and fail to meet real-time business demands[23,24]. Recent AI/ML-based elastic allocation algorithms predict demand dynamically via historical data[25] but struggle with accurate prediction and efficient allocation in complex environments. Deep learning-based reservation algorithms lack robustness in extreme scenarios (sudden traffic surges, equipment failures), leading to operational disruptions from insufficient reservation or resource wastage from excessive allocation[26]. Reinforcement learning-based models optimize strategies through environmental interaction[27,28] but face two key challenges: designing reward functions that account for resource types, task priorities, and real-time needs; and avoiding local optima during training[29].

Resource Management Challenges in Computing Power Networks

Traditional single-dimension resource allocation models have been replaced by dual-dimension models based on resource usage intensity and time periods. These models leverage time series analysis to predict demand and adjust allocation strategies[30–32] but suffer from imprecise user behavior prediction, undermining demand forecasting accuracy and allocation effectiveness.

To address business volume fluctuations, dynamic usage-time based allocation models adopt dynamic programming algorithms, integrating real-time usage and time variables to construct optimal strategies[33–35]. However, high computational complexity increases implementation difficulty, limiting large-scale application. In allocation-reservation coordination, game theory-based models face convergence issues due to complex user demands and information asymmetry[36]. Reinforcement learning-based models are also applied, but computing power network complexity exacerbates inherent challenges (long training times, local optima), hindering practical deployment[37]. Cross-cloud resource management frameworks using blockchain enable

ARTICLE IN PRESS

tamper-proof resource recording[38,39]. Their scalability is limited by two factors: significant differences in cloud providers' resource characteristics and service quality; and blockchain performance bottlenecks (e.g., low throughput) that fail to meet real-time requirements.

Furthermore, unresolved critical challenges remain. Deep learning in computing power scheduling is limited by multimodal data feature extraction, restricting accuracy in complex scenarios[40]. Reinforcement learning-game theory integration faces Nash equilibrium solving challenges for multi-task allocation, with communication overhead reducing algorithm convergence speed and stability[41]. Data augmentation and transfer learning alleviate reinforcement learning data sparsity but lack sufficient effectiveness, while evolving network attacks reduce traditional security measure efficacy, posing additional resource management risks.

Research Innovations

This study proposes the TSL-HRL model architecture, which innovatively combines TSMixer, LDS, and Hierarchical RL to construct an elastic resource reservation algorithm system for computing power networks. The TSMixer module adopts a hybrid expert architecture with a dynamic gating mechanism, selecting the most suitable expert module based on the input time series data characteristics to precisely extract periodic patterns and burst changes in resource usage data. The LDS module uses state-space models and Kalman filtering techniques to estimate resource demand in real-time, dynamically adjusting model parameters to effectively characterize resource demand dynamics in complex network environments. The Hierarchical RL module builds a layered decision-making system where the upper layer uses Q-learning to establish long-term resource allocation frameworks, while the lower layer, based on A2C algorithms, fine-tunes strategies according to real-time network status and user needs, achieving long- and short-term collaborative optimization of decisions.

Based on the above methods, this research offers the following three core contributions:

- The TSL-HRL model, through multi-module collaboration, can more comprehensively mine resource usage data features, accurately capturing dynamic resource demand changes. Compared to traditional time series prediction models, it achieves higher accuracy in computing power network resource demand prediction, providing reliable data support for resource reservation.
- In response to the high dynamics and uncertainty of computing power networks, the model's modules work closely together to quickly perceive network status changes. When unexpected traffic surges, device failures, or other issues occur, the model can promptly adjust resource reservation strategies, showing stronger adaptability in dynamic environments compared to existing algorithms, ensuring efficient network resource allocation.
- The TSL-HRL model uses multi-objective optimization based on resource utilization, task completion efficiency, and service quality, finding the optimal balance for each objective in dynamic environments through a hierarchical reinforcement learning strategy. This not only improves resource utilization efficiency but also ensures stable service delivery, providing a more practical solution for computing power network resource management.

Methods

Architecture Design Concept and Framework

In the context of highly dynamic and complex multi-objective characteristics of computing power network resource demands, traditional models fail to meet the precise resource reservation requirements. The TSL-HRL model is designed to address this challenge, with its architecture closely centered around two core principles: dynamic adaptability and multi-objective optimization. The goal is to achieve efficient resource allocation. Figure 1 illustrates the overall architecture of the TSL-HRL model, with dynamic adaptability and multi-objective optimization as its core focus.

The design concept of the TSL-HRL model stems from a deep analysis of the pain points in computing power network resource management. Faced with sudden changes in resource demand, such as traffic surges during e-commerce promotions,

ARTICLE IN PRESS

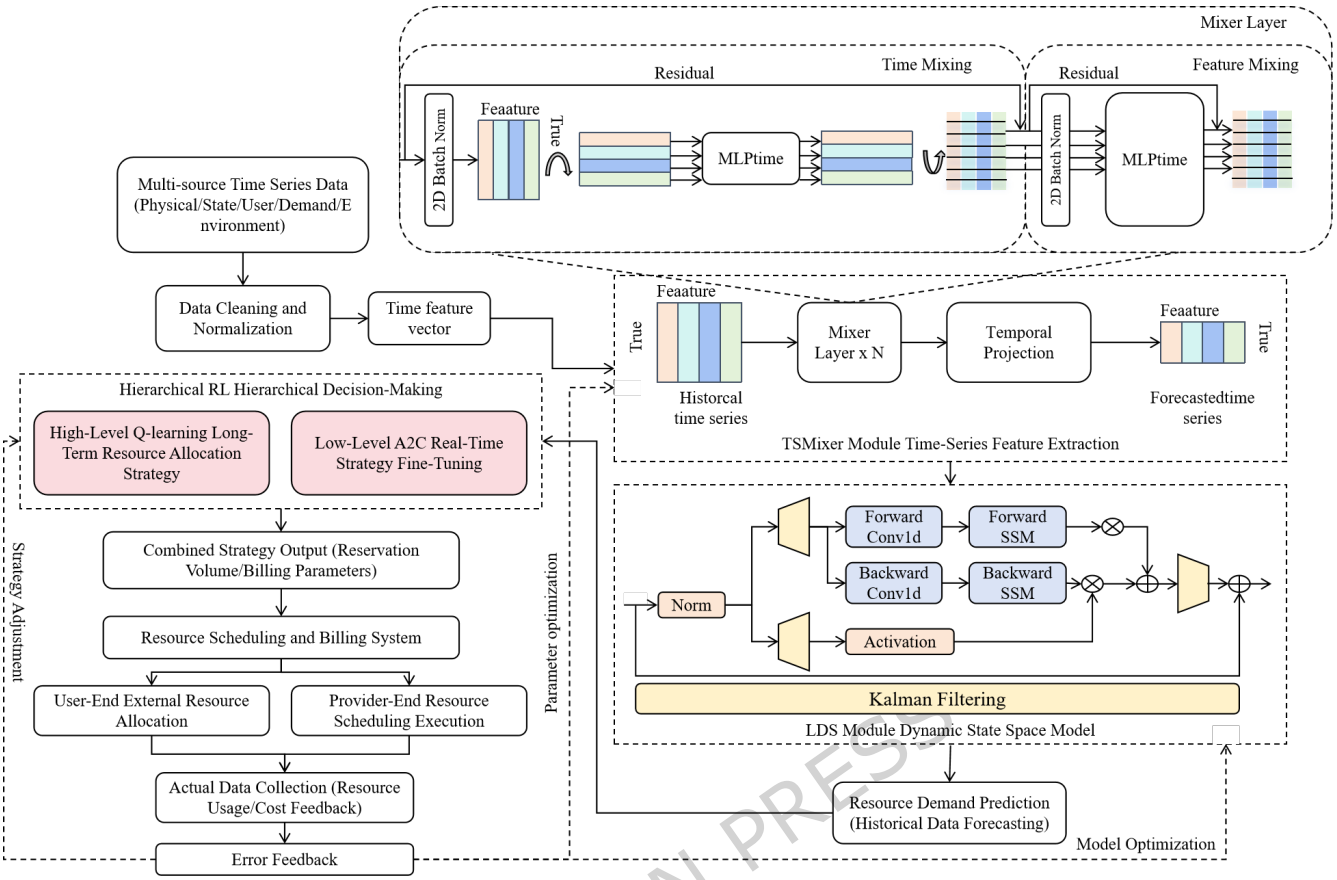

Figure 1. The TSL-HRL model overall architecture. This architecture diagram systematically presents the full-process framework of the computing power network elastic resource reservation model, which is composed of three core modules: TSMixer, LDS, and Hierarchical RL (abbreviated as TSL-HRL). Multi-source time-series data (such as load and user demand) is cleaned and normalized before being input into the TSMixer module to extract periodic fluctuations and burst demand features. These features are then passed to the LDS module, where resource demand is dynamically estimated using a state-space model and Kalman filtering. Subsequently, the Hierarchical RL module uses high-level Q-learning to formulate long-term reservation strategies, while the low-level A2C algorithm performs real-time fine-tuning, collaboratively outputting resource reservation schemes.

ARTICLE IN PRESS

and the complexity of managing multiple objectives—such as resource utilization, task completion efficiency, and service quality—the model is guided by dynamic adaptability, ensuring real-time perception of network state changes and quick strategy adjustments. Additionally, driven by multi-objective optimization, the model seeks the optimal balance among different objectives during resource reservation decisions. The model is composed of three core modules that operate synergistically: TSMixer, LDS, and Hierarchical RL. After data enters the input layer, the TSMixer module extracts deep features from the time series data, uncovering periodic patterns and burst changes in resource usage. Subsequently, the LDS module, based on the features output by TSMixer, utilizes state-space models and Kalman filtering techniques to accurately model the dynamic evolution of resource demand. Finally, the Hierarchical RL module uses the resource demand prediction results from the LDS module, combined with real-time network status and user needs, to develop resource reservation strategies through a layered decision-making mechanism. These three modules work seamlessly together, with information being passed and feedback given between them in an orderly fashion, forming an efficient operating system that provides an innovative and effective solution for elastic resource reservation in computing power networks.

ARTICLE IN PRESS

TSMixer Temporal Data Processing Module

In the complex resource management scenarios of computing power networks, accurate feature extraction from temporal data serves as a crucial prerequisite for achieving efficient resource reservation. Traditional temporal data processing methods struggle to comprehensively mine multi-dimensional features—such as periodicity, trends, and sudden changes—contained in the data, which results in resource management decisions lacking precise data support. The TSMixer module addresses this by using a lightweight design: it combines a mixture-of-experts architecture, two mixing MLPs (temporal and feature), and a gating attention mechanism to efficiently extract key temporal features, avoiding the incomplete feature mining of traditional methods. Its core goal is to provide high-quality feature information for subsequent modules, supporting rational decisions in dynamic networks.

The TSMixer module adopts a mixture-of-experts architecture to deeply extract temporal data features. When the input temporal data X enters the module, it is first analyzed by the gating network based on data characteristics. The gating network assigns dynamic weights (g_i) to N expert modules: modules more relevant to the current data get higher weights, ensuring targeted feature extraction. Assuming the weight vector output by the gating network is G, where its element g_i represents the weight corresponding to the i-th expert module, the calculation formula is expressed as:

$$g_i = \frac{\exp(s_i)}{\sum_{j=1}^{N} \exp(s_j)} \tag{1}$$

Here, s_i denotes the score assigned to the i-th expert module by the gating network, and N stands for the total number of expert modules. Through this formula, the weight g_i is dynamically adjusted according to the characteristics of the input data, ensuring that expert modules highly relevant to the current data are activated.

In the temporal mixing stage, let X_t be the input of the temporal mixing MLP (i.e., the part of the temporal data processed by the gating network in the time dimension), with a shape of $[B, T, F]$ (B = batch size, T = number of time steps, F = number of features). This MLP focuses on the time dimension: it uses a fully connected layer to capture how resource features evolve over time. The temporal mixing MLP processes X_t: first, it passes through a fully connected layer. Let W_t be the weight matrix of the fully connected layer and b_t be the bias; the output is calculated as:

$$Y_t = \text{Activation}(X_t \cdot W_t + b_t) \tag{2}$$

In this equation, Activation refers to an activation function (e.g., the ReLU function). This operation enables the modeling of temporal patterns within the time series.

In the feature mixing stage, let the input of the feature mixing MLP be the output of Y_t after temporal mixing. This MLP is shared across time steps and focuses on feature interactions: it uses transpose operations (Y_t^T) to fuse correlations between different resources (e.g., CPU and memory usage). The feature mixing MLP is shared across time steps and is used to mine covariate information. Let W_f be its weight matrix and b_f be the bias; the output is given by:

$$Z = \text{Activation}(Y_t^T \cdot W_f + b_f)^T \tag{3}$$

The transpose operation allows the feature mixing MLP to effectively fuse information among different features.

To further highlight important features, a Gating Attention (GA) mechanism is added after the MLP block of each mixing component.GA uses an attention weight matrix (A) to weight the MLP output (Z): it amplifies key features (e.g., burst traffic signals) and reduces the impact of redundant ones, improving feature quality. Let Z be the input of the GA module and Z' be the output; then:

$$Z' = Z \cdot A \tag{4}$$

ARTICLE IN PRESS

Here, A is the attention weight matrix, and its element a_{ij} is obtained by calculating the input features. It amplifies major features and reduces the impact of unimportant features in a probabilistic manner, achieving effective feature screening.

Finally, after a series of processing steps,a Dropout operation is used to randomly deactivate some neurons (preventing overfitting), resulting in the final feature representation (O) that integrates temporal evolution, feature correlations, and key feature enhancements. The output feature representation O is:

$$O = \text{Dropout}(Z') \tag{5}$$

The Dropout operation randomly sets the output of some neurons to zero to prevent model overfitting, yielding the final output after the TSMixer module deeply extracts features. This output provides strong data support for the subsequent LDS module to model the dynamic evolution of resources. It covers the evolutionary laws of resource usage in the time dimension, the correlations between different resource features, and the enhancement of key features. This effectively compensates for the shortcomings of traditional methods in feature extraction, significantly improves the model's ability to understand and process complex temporal data, and lays a solid foundation for the accuracy and reliability of the computing power network resource management model.

LDS Dynamic Evolution Modeling Module

The LDS module uses state space models and Kalman filtering to model how resource demand in computing power networks changes dynamically—including nonlinear fluctuations and random disturbances. It takes TSMixer's feature output as input and describes resource demand evolution clearly through two core parts: state equations (tracking hidden state changes) and observation equations (linking hidden states to actual observations).

The LDS module constructs a mathematical modeling framework for the dynamic evolution of resource demand based on state space models and Kalman filtering technology, aiming to accurately capture the nonlinear changes and random disturbances of resource demand over time in computing power networks. This module takes the feature representation O output by TSMixer as input, and achieves a refined description of the dynamic process of resource demand through the synergy of state equations and observation equations.

Let the hidden state of resource demand be h_t (this state reflects unobserved but key characteristics of resource demand).The state equation describes how h_t evolves from the previous moment ($t-1$), including three parts: the natural transition of the state, the influence of external control, and random system disturbances. The state at time t evolves from the state at time $t-1$, and the state equation can be expressed as:

$$h_t = A_t \cdot h_{t-1} + B_t \cdot u_t + w_t \tag{6}$$

where A_t is the state transition matrix, B_t is the control input matrix, u_t is the external control variable, and w_t is Gaussian noise with mean 0 and covariance Q_t, describing random disturbances inside the system.

The observation equation connects the hidden state h_t to the actual observed resource demand (y_t), accounting for small errors that occur during observation. There is a linear mapping relationship between the observed value y_t of resource demand and the hidden state h_t. The observation equation is defined as:

$$y_t = C_t \cdot h_t + D_t \cdot u_t + v_t \tag{7}$$

where C_t is the observation matrix, D_t is the direct transmission matrix, and v_t is Gaussian noise with mean 0 and covariance R_t, representing errors in the observation process.

Kalman filtering works in two steps: first, predicting the "prior" state of t using the optimal state of $t-1$; second, correcting this prediction with the actual observation y_t. The prediction phase below calculates the initial estimate of h_t and its uncertainty (covariance). In the prediction phase of Kalman filtering, based on the optimal estimation $h_{t-1|t-1}$ at time $t-1$, the prior

ARTICLE IN PRESS

estimation of the state at time t is performed:

$$h_{t|t-1} = A_t \cdot h_{t-1|t-1} + B_t \cdot u_t \tag{8}$$

The corresponding covariance matrix is:

$$P_{t|t-1} = A_t \cdot P_{t-1|t-1} \cdot A_t^T + Q_t \tag{9}$$

where $P_{t|t-1}$ is the prior estimation covariance, and $P_{t-1|t-1}$ is the posterior estimation covariance at time $t-1$.

The update phase starts with calculating the Kalman gain: this value determines how much weight to give the actual observation y_t when correcting the prior state (balancing the reliability of prior estimates and observations). In the update phase, the prior estimation is corrected by combining the observed value y_t at time t, and the Kalman gain K_t is calculated:

$$K_t = P_{t|t-1} \cdot C_t^T \cdot (C_t \cdot P_{t|t-1} \cdot C_t^T + R_t)^{-1} \tag{10}$$

This gain reflects the correction weight of the observed value on the state estimation, balancing the influence of prior knowledge and observation data.

Finally, the prior state is corrected using y_t and K_t to get the "posterior" state ($h_{t|t}$)—the most accurate estimate of h_t—and its updated uncertainty. Finally, the posterior state estimation $h_{t|t}$ and its covariance $P_{t|t}$ at time t are obtained:

$$h_{t|t} = h_{t|t-1} + K_t \cdot (y_t - C_t \cdot h_{t|t-1} - D_t \cdot u_t) \tag{11}$$

$$P_{t|t} = (I - K_t \cdot C_t) \cdot P_{t|t-1} \tag{12}$$

where I is the identity matrix. Through the above recursive process, the LDS module can track the dynamic evolution trajectory of resource demand in real time, provide accurate resource state prediction for the Hierarchical RL module, significantly improve the model's adaptability to sudden traffic and dynamic changes in computing power networks, and lay a quantitative foundation for the formulation of elastic resource reservation strategies.

ARTICLE IN PRESS

Hierarchical RL Strategy Optimization Module

The Hierarchical RL module uses a two-layer decision structure to optimize resource reservation in computing power networks: the high-level strategy sets long-term resource allocation frameworks, while the low-level strategy makes real-time scheduling adjustments under the high-level constraints. This collaboration balances long-term stability and short-term adaptability.

The Hierarchical RL strategy optimization module adopts a hierarchical decision-making architecture, achieving dynamic optimization of resource reservation strategies in computing power networks through collaborative optimization of high-level and low-level strategies. This module takes the resource state estimation output by LDS as input, constructing a decision-making mechanism that combines long-term goals with short-term adjustments.

The high-level strategy's action depends on its current macro state and is controlled by parameters θ^H. It outputs a resource allocation framework to guide the low-level strategy.Let the state of the high-level strategy be S^H. The action A^H at time t is determined by the strategy π^H, i.e.:

$$A_t^H = \pi^H(S_t^H; \theta^H) \tag{13}$$

where θ^H are the high-level strategy parameters, S_t^H contains macro information such as long-term resource planning goals and cumulative task completion rate, and A_t^H represents the formulation of resource allocation frameworks.

ARTICLE IN PRESS

The high-level reward balances two long-term goals: resource utilization (how efficiently resources are used) and task completion rate (how many tasks are finished), with weights α and β adjusting their importance. The reward function R^H of the high-level strategy comprehensively considers long-term resource utilization and service quality goals, defined as:

$$R_t^H = \alpha \cdot U_t + \beta \cdot Comp_t \tag{14}$$

where U_t is the resource utilization rate at time t, $Comp_t$ is the cumulative task completion rate, and α and β are weight coefficients balancing resource efficiency and service quality goals.

The low-level strategy operates under the high-level framework (A_t^H), using micro-state information (e.g., real-time load, user request queues) to output real-time scheduling actions, controlled by parameters θ^L. The low-level strategy takes A_t^H output by the high-level strategy as constraints. Its state S_t^L contains micro information such as real-time resource load and user request queues, and the action A_t^L is determined by the strategy π^L:

$$A_t^L = \pi^L(S_t^L, A_t^H; \theta^L) \tag{15}$$

where θ^L are the low-level strategy parameters, and A_t^L realizes real-time adjustment of resource scheduling.

The low-level reward focuses on short-term service quality: it penalizes task delay and rewards fair resource allocation, with weights γ and δ prioritizing these two factors. The immediate reward R^L of the low-level strategy focuses on short-term response effects, calculated as:

$$R_t^L = \gamma \cdot (1 - Delay_t) + \delta \cdot Fair_t \tag{16}$$

where $Delay_t$ is the task processing delay rate, $Fair_t$ is the resource allocation fairness index, and γ and δ are weights prioritizing service quality and allocation fairness.

The high-level strategy optimizes by maximizing the sum of discounted long-term rewards—this ensures decisions align with long-term goals, where γ reduces the impact of distant, less relevant rewards. The high-level strategy updates parameters through cumulative discounted rewards, with its objective function J^H as:

$$J^H(\theta^H) = E\left[\sum_{t=0}^{T} \gamma^t R_t^H \mid \pi^H\right] \tag{17}$$

where γ is the discount factor balancing the impact of recent and long-term rewards.

The low-level strategy uses the Advantage Actor-Critic (A2C) algorithm: the advantage function A_t^L measures how much better the current action is than the average action (by comparing the current reward + future state value to the current state value), guiding more effective parameter updates. The low-level strategy is optimized using the Advantage Actor-Critic algorithm, with the advantage function A_t^L defined as:

$$A_t^L = R_t^L + \gamma \cdot V^L(S_{t+1}^L) - V^L(S_t^L) \tag{18}$$

where V^L is the value function evaluating state value, and A_t^L measures the advantage of actions relative to the average level.

To intuitively present the interaction flow, update frequency, convergence judgment, exploration strategy, target network updates and training mode between the high-level Q-learning and low-level A2C strategies, the core algorithm 1 pseudocode is given below:

Both layers update parameters via gradient descent: the high-level strategy maximizes its long-term reward objective, while the low-level strategy maximizes the advantage of its actions—this alternating optimization refines the overall strategy. Parameters are updated through hierarchical policy gradient descent, with high-level and low-level strategies optimized

ARTICLE IN PRESS

Algorithm 1: High-Level Q-Learning & Low-Level A2C Interaction

Input : Initial params: θ^H (Q-learning), θ_a^L (A2C actor), θ_c^L (A2C critic);
Q-learning update freq: $T = 10$; Convergence thr: $\varepsilon = 1e-4$; Max episodes: $N = 2500$;
Exploration: $\varepsilon_{init} = 0.9$, $\varepsilon_{final} = 0.05$, decay step = 100; Target update rate: $\tau = 0.01$
Output : Optimized θ^H, θ_a^L, θ_c^L
Initialization: $t = 0$, iter = 0, $Loss_{total} = \infty$, $\varepsilon = \varepsilon_{init}$,
target $\theta_{target}^H = \theta^H$, target $\theta_{c,target}^L = \theta_c^L$;
while $iter < N$ && $Loss_{total} > \varepsilon$ **do**
  if $t \mod T == 0$ **then**
    $S_t^H \leftarrow$ [Long-term goal, Cumulative task completion rate];
    $A_t^H \leftarrow$ Random resource allocation framework (if rand() < ε) else $\pi^H(S_t^H;\theta^H)$;
    $R_t^H \leftarrow \alpha U_t + \beta Comp_t$;
    $\theta^H \leftarrow \arg\max J^H(\theta^H)$;
    $\theta_{target}^H \leftarrow \tau \cdot \theta^H + (1-\tau) \cdot \theta_{target}^H$;
  end
  $S_t^L \leftarrow$ [Real-time load, User queue, Task priority];
  $A_t^L \leftarrow$ Random scheduling action (if rand() < ε) else $\pi^L(S_t^L, A_t^H; \theta_a^L)$;
  $R_t^L \leftarrow \gamma(1 - Delay_t) + \delta Fair_t$;
  $A_t^L \leftarrow R_t^L + \gamma \cdot V^L(S_{t+1}^L; \theta_{c,target}^L) - V^L(S_t^L; \theta_c^L)$;
  $\theta_a^L, \theta_c^L \leftarrow$ Update via advantage A_t^L;
  $\theta_{c,target}^L \leftarrow \tau \cdot \theta_c^L + (1-\tau) \cdot \theta_{c,target}^L$;
  $Loss_{total} = Loss^H + Loss^L$;
  $\varepsilon \leftarrow \max(\varepsilon_{final}, \varepsilon - 0.1)$ **if** iter $\mod$ 100 == 0 **else** ε;
  $t++, iter++$;
end

alternately, ultimately achieving:

$$\theta^{H*} = \arg\max J^H(\theta^H) \tag{19}$$

$$\theta^{L*} = \arg\max E\left[\sum_{t=0}^{T} A_t^L \cdot \log \pi^L(A_t^L \mid S_t^L, A_t^H; \theta^L)\right] \tag{20}$$

This hierarchical optimization mechanism not only ensures long-term stability of resource management but also quickly responds to network dynamic changes, improving resource utilization while achieving multi-objective balance between service quality and task completion efficiency.

Experiment

Experimental Objectives and Scenario Design

This experiment aims to comprehensively validate the effectiveness of the TSL-HRL model in computing power network resource management based on the Multi-Cloud Service Composition Dataset and the Google 2019 Cluster sample dataset, with a focus on three core performances: dynamic resource reservation accuracy, multi-objective optimization capability, and adaptability under high concurrency.

In real applications, computing power networks often face challenges such as sudden traffic surges, complex business types, and dynamic changes in nodes. Accordingly, three representative experimental scenarios are designed. The sudden traffic scenario, based on the Google 2019 Cluster sample dataset, simulates short-term surges in resource demand such as those

ARTICLE IN PRESS

occurring during e-commerce promotions and live-streaming events. In this dataset, CPU utilization has been observed to peak at 5–8 times the average within certain periods. The experiment will reproduce such surges to test whether the TSL-HRL model can quickly and accurately sense traffic changes, promptly adjust resource reservation strategies, and avoid service interruptions caused by insufficient resources or waste due to over-reservation.

The multi-cloud service composition scenario is based on the Multi-Cloud Service Composition Dataset, which contains over 1,000 cases of multi-cloud service compositions, including computing, storage, and network services, with computing accounting for 35%, storage 25%, and networking 40%. The experiment simulates enterprises running diverse workloads such as AI model training, data backup, and real-time analytics in a multi-cloud environment, evaluating how the TSL-HRL model can reasonably allocate and schedule different types of resources under complex business demands to ensure service quality across applications while improving overall resource utilization efficiency.

The cross-cloud node dynamic scenario combines the features of the two datasets, simulating dynamic changes in multi-cloud node topology. The Google 2019 Cluster sample records node offline and addition events. In the experiment, the node change rate per event is set at no more than 15% of total nodes, simulating node failures due to equipment or network issues and new node additions caused by business expansion. Under this dynamic environment, the TSL-HRL model is tested for its ability to update resource state awareness in real time, flexibly adjust resource reservation strategies through the hierarchical decision-making mechanism of the Hierarchical RL module, maintain stable network operations, and balance the interests of all parties, thus verifying its adaptability and robustness in complex and changing network environments.

Experimental Data

The experimental data preprocessing is based on the Multi-Cloud Service Composition Dataset and the Google 2019 Cluster sample, involving cleaning, transformation, and augmentation operations to construct structured data directly usable by the model. Key features of these two datasets, including node quantity, resource metrics, time span, and sampling interval, are summarized in Table 1 for intuitive reference.

ARTICLE IN PRESS

Table 1. Summary of Dataset Features.

Feature	Google 2019 Cluster Sample	Multi-Cloud Service Composition Dataset
Number of Nodes/Clusters	8 compute clusters	200 multi-cloud nodes
Resource Metrics	CPU usage histograms, alloc set info, job parent-child relationships, resource requests	CPU utilization, memory usage, service response time, network latency
Time Span	1 month (May 2019)	7 days
Sampling Interval	5 minutes	1 minute
Data Volume	1,382,400 records	2,016,000 records
Application Scenario	Cluster scheduling, cloud computing research	Multi-cloud service composition optimization

For the Multi-Cloud Service Composition Dataset, the core fields are first extracted from over 1000 multi-cloud service composition cases, including service types (computing, storage, networking), resource demand sequences (CPU, memory, and storage usage recorded every 15 minutes), service response delay, and user cost labels. Since the original data contains 5% missing values (mainly in the storage IOPS and network bandwidth fields), a K-nearest neighbor interpolation method based on business types is applied to fill in the missing data—i.e., missing data points are filled based on resource usage patterns of similar services, preventing feature distortion caused by simple mean imputation. Additionally, 3% of abnormal records (such as extreme service response delays more than 10 times the average) are filtered out, leaving only samples that align with actual business patterns.

For the Google 2019 Cluster sample, the focus is on processing 2 million cluster resource usage records, extracting metrics such as CPU utilization, memory usage, and node status (online/offline) at a 1-minute granularity. Given the large time span of the original data (covering 30 days), a sliding window method (window size 1 hour) is used to extract segments containing burst traffic features (e.g., CPU utilization rising from 30% to 90%), and node dynamic change events (such as offline and

ARTICLE IN PRESS

newly added nodes, along with the time and duration of these events) are marked. To match the model input dimensions, the 1-minute granularity data is resampled to 10-second intervals (consistent with the experiment's scheduling cycle), with linear interpolation used to maintain data continuity.

Both datasets undergo standardization to eliminate dimensional differences: for ratio-based indicators like CPU utilization and memory usage, Min-Max normalization is applied to map the values to the range [0,1]; for absolute values such as service delay and resource demand, Z-score normalization is used (subtracting the mean and dividing by the standard deviation) to ensure the data distribution meets the requirements for model training. Finally, the datasets are split into training, validation, and test sets at a 7:2:1 ratio—70% of the regular samples and 20% of the burst scenario samples are used for training, the validation set is used for tuning hyperparameters (such as the number of expert modules in TSMixer and the learning rate in RL), and the test set, containing 30% of the burst traffic and node dynamic change samples not involved in training, ensures the objectivity of the evaluation results. To confirm no data leakage in the Google 2019 Cluster sample: 1) Timestamp consistency verification is performed, confirming that the time ranges of training, validation, and test sets are completely non-overlapping and follow chronological order; 2) Causal feature validation is conducted to ensure all input features are derived from historical data before time t, with no future information involved—this is enforced by locking the feature extraction time window to $t-30$ to t during data preprocessing.

Experimental Detail Setup

Hardware and Software Configuration

The experiment adopts a multi-cloud simulation framework, using Docker containers to simulate five heterogeneous cloud nodes (three AWS instances and two Alibaba Cloud instances). In terms of hardware, each cloud node is equipped with an 8-core Intel Xeon E5-2680 v4 CPU, 32 GB of memory, 1 TB SSD storage, and an NVIDIA A100 GPU (40GB VRAM)—ensuring reproducibility of model training and inference. In terms of software, the CloudSim Plus framework is employed to build the multi-cloud environment, combined with Kubernetes for containerized resource scheduling, and the model training and evaluation platform is built on Python 3.8.

Key Parameter Settings

Network environment parameters are configured according to the real topological features of the Multi-Cloud Service Composition Dataset. Cross-cloud link bandwidth ranges from 1 to 10 Gbps, with 10 Gbps between core nodes and 1 Gbps for edge node access. Cross-cloud link latency is controlled within 10 to 50 ms, with 10 to 20 ms between AWS instances and 30 to 50 ms between AWS and Alibaba Cloud instances. Random jitter of ±5 ms is added to simulate real network fluctuations.

Resource monitoring focuses on three core metrics: CPU capacity in cores, memory bandwidth in GB/s, and storage IOPS in operations per second. These metrics correspond to the main monitoring dimensions of the Google dataset for direct comparative analysis. For resource scheduling constraints, the maximum resource utilization threshold per node is set to 85% to prevent overload and service degradation. Cross-cloud resource migration time is capped at 2 seconds to ensure scheduling flexibility. Single-node resource adjustments are limited to no more than 20% of total capacity per adjustment to maintain system stability. Business load parameters are configured based on service characteristics from the Multi-Cloud dataset. AI training tasks require 2 to 8 CPU cores and 8 to 32 GB memory per instance. Data backup tasks require 500 to 2000 IOPS. Real-time analytics tasks require 100 to 500 Mbps of network bandwidth. The concurrency ratio of these three tasks is set at 3:2:5, reflecting the distribution in the dataset to ensure consistency between the experimental load and real-world scenarios.

Key hyperparameters of the TSL-HRL model: TSMixer mixing layer channels = 64; optimizer is AdamW with learning rate = 0.001; in the hierarchical RL module, high-level Q-learning update interval $T = 10$, low-level A2C discount factor = 0.95, replay buffer capacity = 10,000, high-level reward weights $\alpha = 0.6$, $\beta = 0.4$, low-level reward weights $\gamma = 0.7$, $\delta = 0.3$, exploration adopts decaying ε-greedy strategy with initial $\varepsilon = 0.9$, final $\varepsilon = 0.05$, decay step = 100, target network uses soft updates with $\tau = 0.01$; all weights and RL training parameters are calibrated via Bayesian optimization. Hyperparameters of additional baselines: PPO-Transformer uses 6 Transformer layers, 64 PPO batch size, Adam optimizer with learning

ARTICLE IN PRESS

rate = 0.0008; GA-RL hybrid framework uses 50 GA population size, 80 generation count, RL learning rate = 0.0005; ARIMA+Q-learning uses ARIMA parameters $p = 3$, $d = 1$, $q = 4$ and Q-learning learning rate = 0.005, discount factor = 0.9; LSTM+DDPG uses 2 LSTM layers, 128 hidden units per layer, DDPG actor/critic learning rate = 0.0003/0.001; GA-PSO uses 60 GA population size, 100 generations, 40 PSO particles, 80 iterations.

Unified experimental constraints for all models: Hyperparameters are optimized via 50 Bayesian optimization trials; 5 independent random seeds are used to reduce randomness; early-stopping is applied—training stops if validation loss shows no improvement for 15 consecutive epochs; two-tailed t-tests are conducted for significance verification, with $p < 0.05$ indicating statistically significant differences.

Model Training Process Pseudocode

To standardize the training execution logic for both the TSL-HRL model and all baseline models, while covering multi-random-seed control, early-stopping mechanism, and other constraints, the algorithm 2 below highlights the core training workflow in the experiment:

Algorithm 2: Unified Model Training Workflow

```
Input  : Bayesian-optimized params (TSL-HRL/baselines); Train/val sets; 5 random seeds; Early-stop thr=15
Output : Best-trained models; Validation metric records
Initialize models (TSL-HRL + baselines) with optimized hyperparameters;
for each random seed in seed set do
    Set random seed for reproducibility;
    for each model in {TSL-HRL, baselines} do
        no_improve = 0, best_val = initial high value;
        while no_improve < 15 do
            Sample training batch → Forward pass (TSL-HRL: TSMixer→LDS→RL; Baselines: per mechanism);
            Update model parameters via optimizer;
            Compute validation metric on val set;
            if current val metric < best_val then
                best_val = current val metric; no_improve = 0; Save best model;
            end
            else
                no_improve += 1
            end
            ;
        end
    end
end
```

ARTICLE IN PRESS

Evaluation Metrics

This paper employs a multi-dimensional evaluation metric system to validate the performance of the TSL-HRL model. In terms of prediction accuracy, the Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) are selected to measure the deviation between the predicted and actual resource demand values. For computational efficiency, the evaluation includes metrics such as single inference time, total training time, model parameter count, and GPU memory peak usage to assess the computational overhead and engineering adaptability of the model. In resource scheduling capability, the evaluation focuses on CPU utilization, storage utilization, and network bandwidth utilization to reflect resource usage efficiency, combined with cross-cloud scheduling latency to assess real-time response capability. For dynamic adaptability, the evaluation quantifies the model's stability and response speed in scenarios with dynamic node changes, using metrics such as service interruption rate, node offline/new addition recovery adaptation time, and resource utilization fluctuation.

ARTICLE IN PRESS

Results

The experiment selects various algorithms as comparison models, including Zhan's LSTM-based edge computing elastic scheduling algorithm[42], Zi's RNN cloud resource load prediction algorithm[8], ALMAZROI's[43] and NGT's[44] multi-task resource management frameworks, Cao's elastic resource scheduling algorithm[25], Lan's knowledge distillation-based elastic scaling algorithm[45], traditional ARIMA+Q-learning[46], LSTM+DDPG[47], heuristic GA-PSO[48], Transformer+PPO[49]. These models cover classic prediction-scheduling integration, deep learning-RL hybrids, heuristic optimization, and state-of-the-art resource management paradigms, with ARIMA+Q-learning representing traditional phased schemes and GA-PSO typical multi-objective heuristic methods, to comprehensively validate the TSL-HRL model's effectiveness.

Table 2. Performance comparison of models on Multi-Cloud Service Composition Dataset and Google 2019 Cluster sample.

Model	Multi-Cloud Service Composition Dataset		Google 2019 Cluster sample	
	MAE	RMSE	MAE	RMSE
TSL-HRL	0.06±0.005*	0.08±0.007*	0.07±0.006*	0.09±0.008*
Zhan	0.11±0.015	0.14±0.018	0.13±0.016	0.16±0.019
Zi	0.12±0.016	0.15±0.019	0.14±0.017	0.17±0.020
ALMAZROI	0.10±0.012	0.13±0.015	0.15±0.014	0.18±0.017
NGT	0.11±0.013	0.14±0.016	0.16±0.015	0.19±0.018
Cao et al.	0.09±0.008	0.12±0.010	0.12±0.011	0.15±0.013
Lan et al.	0.08±0.007	0.10±0.009	0.10±0.008	0.12±0.010
ARIMA+Q-learning	0.17±0.020	0.21±0.023	0.19±0.021	0.23±0.025
LSTM+DDPG	0.07±0.006	0.09±0.008	0.09±0.009	0.11±0.011
GA-PSO	0.13±0.014	0.16±0.017	0.15±0.016	0.18±0.019
Transformer+PPO	0.07±0.006	0.09±0.008	0.08±0.007	0.10±0.009

* Statistically significant improvement (two-tailed t-test, $p < 0.05$) over all baseline models. Key p-values: TSL-HRL vs Transformer+PPO (MAE: 0.032, RMSE: 0.038); TSL-HRL vs LSTM+DDPG (MAE: 0.029, RMSE: 0.035). All p-values are calculated from 5 independent runs.

ARTICLE IN PRESS

The MAE and RMSE performance of each model on the Multi-Cloud Service Composition Dataset and Google 2019 Cluster sample datasets are shown in the table 2. The TSL-HRL model significantly outperforms most of the comparison models in terms of prediction accuracy. For the Multi-Cloud dataset, TSL-HRL has an MAE of 0.06 and an RMSE of 0.08, which is approximately 45% lower than Zhan's MAE of 0.11 and RMSE of 0.14, and more than 50% lower than Zi's MAE of 0.12 and RMSE of 0.15. Even compared to the better-performing Transformer+PPO, which has an MAE of 0.07 and RMSE of 0.09, TSL-HRL reduces the MAE by 14.3% and the RMSE by 11.1%. In the Google dynamic dataset scenario, TSL-HRL's MAE and RMSE are 0.07 and 0.09, respectively, reducing the MAE by 46% and the RMSE by 43.7% compared to Zhan's MAE of 0.13 and RMSE of 0.16. It also shows a reduction of 50% and 47% compared to Zi's MAE of 0.14 and RMSE of 0.17, and clearly outperforms models such as ALMAZROI and NGT. It is worth noting that LSTM+DDPG achieves an MAE of 0.09 and RMSE of 0.11 on the Google dataset, which is close to TSL-HRL but still lags behind by 22.2% and 18.2%, respectively. From the data distribution perspective, TSL-HRL consistently achieves the lowest MAE and RMSE across both datasets. Moreover, the error fluctuations across datasets are significantly smaller, with a 16.7% increase in MAE and a 12.5% increase in RMSE from Multi-Cloud to Google, much smaller than Zhan's MAE increase of 18.2% and RMSE increase of 14.3%, as well as Transformer+PPO's MAE increase of 14.3% and RMSE increase of 11.1%. This indicates that TSL-HRL not only delivers high prediction accuracy but also demonstrates stronger generalization ability across scenarios. It can effectively handle data scenarios with different characteristics, such as multi-cloud heterogeneous environments and dynamic cluster nodes, validating the model's ability to accurately capture periodic and dynamic features in time series data through the integration of the TSMixer and LDS modules.

From the table 3, it can be seen that the TSL-HRL model demonstrates significantly better computational and training

ARTICLE IN PRESS

Table 3. Computational efficiency comparison of models on Multi-Cloud Service Composition Dataset and Google 2019 Cluster sample.

Model	Multi-Cloud Service Composition Dataset				Google 2019 Cluster sample			
	Inference Time (ms)	Training Time (h)	Params (M)	Peak GPU (GB)	Inference Time (ms)	Training Time (h)	Params (M)	Peak GPU (GB)
TSL-HRL	82.5	3.9	12.8	4.5	88.1	4.5	12.8	4.9
Zhan	208.3	9.1	28.6	8.0	223.2	9.9	28.6	8.4
Zi	231.7	10.5	32.1	8.9	245.1	11.1	32.1	9.3
ALMAZROI	182.4	8.3	25.3	7.3	196.8	9.1	25.3	7.7
NGT	197.2	8.8	27.8	7.8	209.8	9.6	27.8	8.2
Cao et al.	150.6	6.5	18.5	6.1	161.8	7.1	18.5	6.5
Lan et al.	124.3	5.6	16.2	5.6	133.1	6.2	16.2	6.0
ARIMA+Q-learning	101.8	3.6	8.5	3.1	109.0	4.0	8.5	3.3
LSTM+DDPG	163.5	7.2	22.4	6.7	175.1	7.8	22.4	7.1
GA-PSO	318.9	11.8	–	5.0	332.3	12.8	–	5.4
Transformer+PPO	137.8	7.8	24.7	7.6	147.2	8.4	24.7	8.0

efficiency on both datasets compared to other models. On the Multi-Cloud Service Composition Dataset, TSL-HRL has an inference time of 82.5 ms, a total training time of 3.9 hours, 12.8M model parameters, and a peak GPU memory usage of 4.5 GB. In comparison, the Zhan model has an inference time of 208.3 ms, a total training time of 9.1 hours, with both parameter count and memory usage significantly higher than TSL-HRL. The Zi model takes even longer with a training time of 10.5 hours and an inference time of 231.7 ms, showing much lower efficiency than TSL-HRL. On the Google 2019 Cluster sample dataset, TSL-HRL's inference time increases to 88.1 ms, and the total training time is 4.5 hours, with a slight increase in model parameters and memory usage, but it still remains at a low level. In contrast, models like Zhan and Zi show larger increases in inference time and training duration. For instance, the Zi model on this dataset has an inference time of 245.1 ms, a total training time of 11.1 hours, and a peak GPU memory usage of 9.3 GB, consuming significantly more resources than TSL-HRL. Although ARIMA+Q-learning performs well in inference time and memory usage, its model parameter count is low, resulting in insufficient prediction accuracy. TSL-HRL, with its efficient architectural design, achieves a balance between low inference time, short training duration, small model size, and low memory usage while maintaining high prediction accuracy. Particularly when handling large-scale dynamic data in the Google dataset, it retains high efficiency and demonstrates good adaptability to different scenarios and practical engineering applications. In comparison, other models either have significant efficiency shortcomings or fail to balance accuracy and efficiency as well as TSL-HRL, fully validating the advantages of TSL-HRL in real-world deployments. TSL-HRL's superiority in accuracy and efficiency stems from its targeted architecture: the LDS module outperforms Transformer/LSTM-based predictors in filtering transient resource load spikes, reducing noise and unnecessary adjustments; its hierarchical RL framework decomposes complex scheduling into sub-tasks, adapting more precisely to dynamics than GA-PSO and single-stage RL models. Computationally, the MLP-based TSMixer and segmented RL updates yield lower complexity than Transformer+PPO, with bounded rewards and segmented learning rates ensuring efficient convergence.

As shown in the figure 2, TSL-HRL demonstrates significantly lower inference times across all batch sizes on both datasets, outperforming models such as Zhan, Zi, and ALMAZROI. Taking the Multi-Cloud dataset as an example, when the batch size is 16, TSL-HRL's inference time is much lower than that of the other models. Even when the batch size increases to 128, the growth in inference time remains relatively gradual, consistently maintaining high inference efficiency. In contrast, models like GA-PSO show a significant increase in inference time as the batch size grows on both datasets, particularly on the Google dataset, where GA-PSO's inference time peaks significantly higher than that of TSL-HRL. This indicates that TSL-HRL, with its architectural advantages, can more efficiently utilize computing resources when handling tasks in multi-cloud service composition and dynamic cluster scenarios. Whether dealing with small or large batch data inference, TSL-HRL maintains low latency and excels in the collaborative optimization of resource scheduling and computational efficiency. Compared to traditional models and other deep learning models, TSL-HRL is better suited to meet the high-dynamic, multi-scenario demands of computing power networks. For the "computing - storage - networking" hybrid service scenario in the Multi-Cloud Service Composition Dataset, the focus is on comparing the resource utilization (reflecting resource waste) and cross-cloud scheduling

ARTICLE IN PRESS

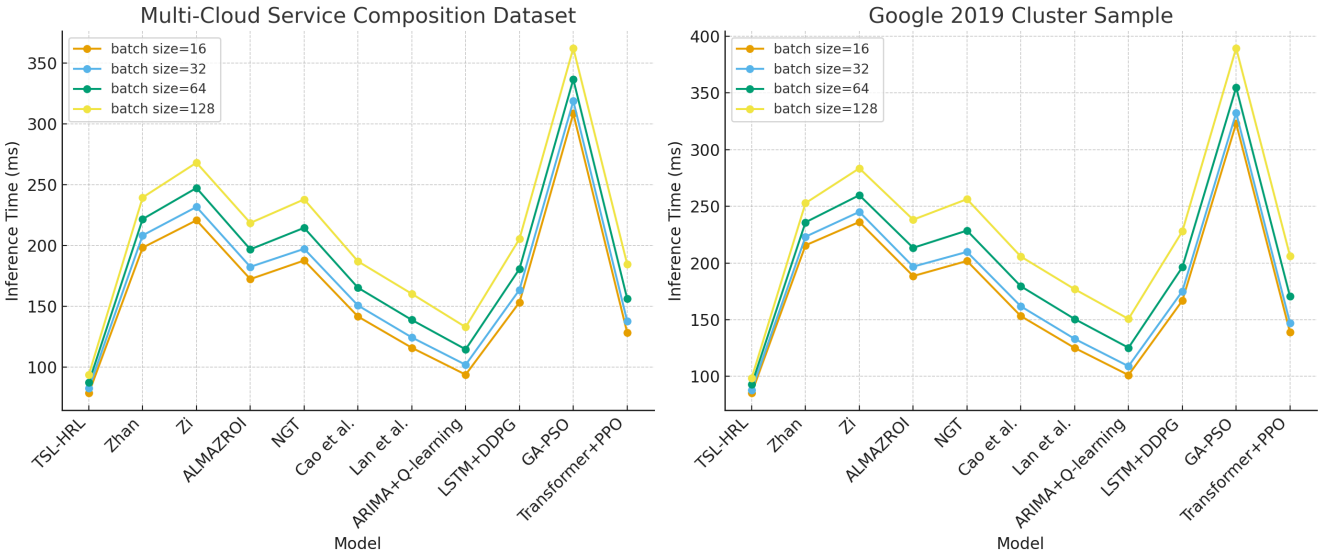

Figure 2. Comparison of inference times across different models with different batch sizes in the Multi-Cloud Service Composition Dataset and Google 2019 Cluster Sample.

latency (reflecting real-time responsiveness) of each model. The results are shown in the table 4 below (the higher the resource utilization and the lower the scheduling latency, the better the performance):

Table 4. Resource Scheduling Efficiency Comparison on Multi-Cloud Service Composition Dataset.

Model	CPU Utilization (%)	Storage Utilization (%)	Network Bandwidth Utilization (%)	Cross-cloud Scheduling Latency (s)	Preliminary Energy Consumption (kWh/h)
TSL-HRL	92.3	88.7	90.5	11.2	18.6
Zhan	78.5	72.3	75.1	75.4	27.3
Zi	76.2	70.1	73.8	68.9	28.5
ALMAZROI	81.4	76.9	79.2	52.7	25.1
NGT	80.1	75.3	77.8	55.3	25.8
Cao et al.	85.7	80.2	83.5	48.6	23.4
Lan et al.	87.9	82.5	85.1	35.2	21.9
ARIMA+Q-learning	69.8	65.4	67.2	82.5	30.7
LSTM+DDPG	88.5	83.1	85.7	42.8	22.7
GA-PSO	77.3	71.8	74.5	63.8	29.1
Transformer+PPO	90.1	86.3	88.2	28.5	20.3

ARTICLE IN PRESS

Table 4 presents the comparison results of resource scheduling efficiency, energy consumption, and other key metrics among different models on the Multi-Cloud Service Composition Dataset. Based on the test settings and evaluation goals, significant differences exist in resource utilization, cross-cloud scheduling latency, and energy efficiency across models. TSL-HRL achieves CPU, storage, and network bandwidth utilization rates of 92.3%, 88.7%, and 90.5%, respectively, far outperforming traditional models. Compared to Zhan's 78.5%, 72.3%, and 75.1%, the average improvement in the three resource utilization metrics exceeds 15%. Even when compared to the cutting-edge Transformer+PPO (90.1%, 86.3%, 88.2%), TSL-HRL still outperforms it by 2.2%, 2.4%, and 2.3% in the three indicators, fully demonstrating its advantage in resource utilization efficiency. This advantage stems from the TSMixer module's precise extraction of resource periodic and burst features, combined with the LDS module's superior filtering of transient load spikes (outperforming Transformer/LSTM-based predictors) to reduce idle waste. In terms of cross-cloud scheduling latency, TSL-HRL performs even better, completing the scheduling in just 11.2 seconds, an 85% reduction compared to Zhan's 75.4 seconds, and a 60.7% reduction compared to the better-performing Transformer+PPO (28.5 seconds), far lower than the traditional ARIMA+Q-learning at 82.5 seconds. Even compared to LSTM+DDPG, a reinforcement learning model (42.8 seconds), the scheduling speed is improved by nearly 74%. This is due to its hierarchical RL framework, which decomposes complex tasks into manageable sub-tasks—outperforming heuristic methods like GA-PSO and single-stage RL models—with upper-level Q-learning providing long-term guidance and

ARTICLE IN PRESS

lower-level A2C enabling real-time fine-tuning. In line with green computing principles, TSL-HRL also achieves superior energy efficiency with a preliminary energy consumption of 18.6 kWh/h, 32.6% lower than ARIMA+Q-learning (30.7 kWh/h) and 8.4% lower than Transformer+PPO (20.3 kWh/h), attributed to optimized resource utilization and reduced idle energy waste. In the complex and dynamic network environment of the Multi-Cloud scenario, TSL-HRL can quickly sense changes in node load, traffic bursts, and other conditions, promptly adjusting resource scheduling strategies, ensuring business continuity while significantly improving the system's real-time responsiveness. The experimental results fully demonstrate that TSL-HRL achieves a new level of balance between resource utilization, scheduling efficiency, and energy conservation, providing a more practical solution for resource management in computing power networks in multi-cloud service scenarios.

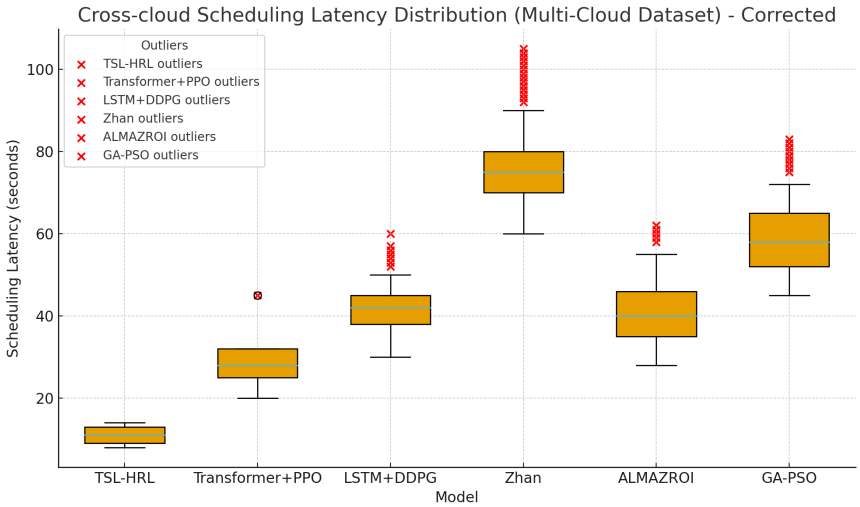

Figure 3. Boxplot of Cross-Cloud Scheduling Latency Distribution for Various Models on the Multi-Cloud Dataset.

This figure 3 illustrates the distribution of cross-cloud scheduling latency for various models on the Multi-Cloud dataset. TSL-HRL performs the best in terms of scheduling latency, with its box plot positioned the lowest, the median concentrated around the 10-second range, and a short box length (small interquartile range), with very few outliers. This indicates that the model not only has low latency but also demonstrates strong consistency and stability in cross-cloud scheduling. In contrast, the median and distribution range of Transformer+PPO's scheduling latency are significantly higher than TSL-HRL's. The latency of LSTM+DDPG is further increased, and the Zhan model exhibits the highest scheduling latency, with its box plot reaching nearly 80 seconds and many extreme outliers, reflecting the model's tendency to experience prolonged delays in multi-cloud scheduling. ALMAZROI and GA-PSO also show broader latency distributions, with their stability and low-latency performance not matching that of TSL-HRL. This disparity arises because TSL-HRL, through hierarchical reinforcement learning and time-series feature modeling, can more accurately perceive the state of multi-cloud resources and make rapid decisions. In contrast, traditional models or single-architecture models struggle to balance decision speed and accuracy in dynamic resource coordination across cloud environments, resulting in higher latency and greater fluctuations.

Based on the "Node Offline / Addition" dynamic scenario in the Google 2019 Cluster sample, the service interruption rate (reflecting stability) and node recovery time (reflecting dynamic adaptability) of each model are compared. The results are shown in the table 5 below (the lower the service interruption rate and the shorter the recovery time, the better the performance):

As shown in the table 5, TSL-HRL demonstrates a dominant advantage in terms of stability and dynamic adaptability. Its service interruption rate is as low as 0.8%, significantly lower than Zhan's 5.3%, Zi's 6.1%, and even outperforming Transformer-based Transformer+PPO (1.5%). This indicates that in a complex environment with frequent node changes, TSL-HRL can minimize the risk of service interruption, ensuring business continuity to the greatest extent. In terms of node offline recovery time, TSL-HRL only requires 45.2 seconds, reducing nearly 75% compared to Zhan's 182.5 seconds, and 28% faster than the next best Transformer+PPO (62.8 seconds). The node addition adaptation time is 38.7 seconds,

ARTICLE IN PRESS

Table 5. Cluster Dynamic Node Adaptability Comparison on Google 2019 Cluster sample

Model	Service Interruption Rate (%)	Node Offline Recovery Time (s)	Node Addition Adaptation Time (s)	Resource Utilization Fluctuation (%)
TSL-HRL	0.8	45.2	38.7	4.2
Zhan	5.3	182.5	165.3	12.7
Zi	6.1	175.8	158.6	13.5
ALMAZROI	4.7	142.3	130.5	11.2
NGT	5.1	150.7	138.2	11.8
Cao et al.	3.2	110.6	98.4	8.5
Lan et al.	2.5	85.3	76.8	6.9
ARIMA+Q-learning	7.8	210.4	195.7	15.3
LSTM+DDPG	2.1	78.5	69.3	6.5
GA-PSO	5.8	168.2	152.4	12.9
Transformer+PPO	1.5	62.8	55.4	5.7

significantly lower than Zi's 158.6 seconds and ALMAZROI's 130.5 seconds, reflecting its ability to rapidly reconfigure resource scheduling strategies. Notably, TSL-HRL has a resource utilization fluctuation of only 4.2%, much lower than Zhan's 12.7% and ARIMA+Q-learning's 15.3%, and even better than LSTM+DDPG, which also employs deep learning (6.5%). This result demonstrates that TSL-HRL, through the TSMixer module's deep feature extraction of resource usage data and the LDS module's dynamic state estimation based on Kalman filtering, can accurately capture the impact of node changes on resource demand. Additionally, the Hierarchical RL decision-making mechanism plays a key role: the upper-level Q-learning algorithm, based on historical data and business priorities, establishes a long-term resource allocation framework to ensure the global nature of the strategy, while the lower-level A2C algorithm fine-tunes the strategy in milliseconds based on real-time network topology, node load, and other information to respond quickly to sudden changes. The synergy between these two components enables TSL-HRL to quickly restore services, adapt resources, and maintain a high level of resource utilization stability in dynamic node change scenarios, effectively solving the core problem of resource management in high-dynamic computing power network environments and validating the advanced and practical nature of its architectural design and algorithmic strategy.

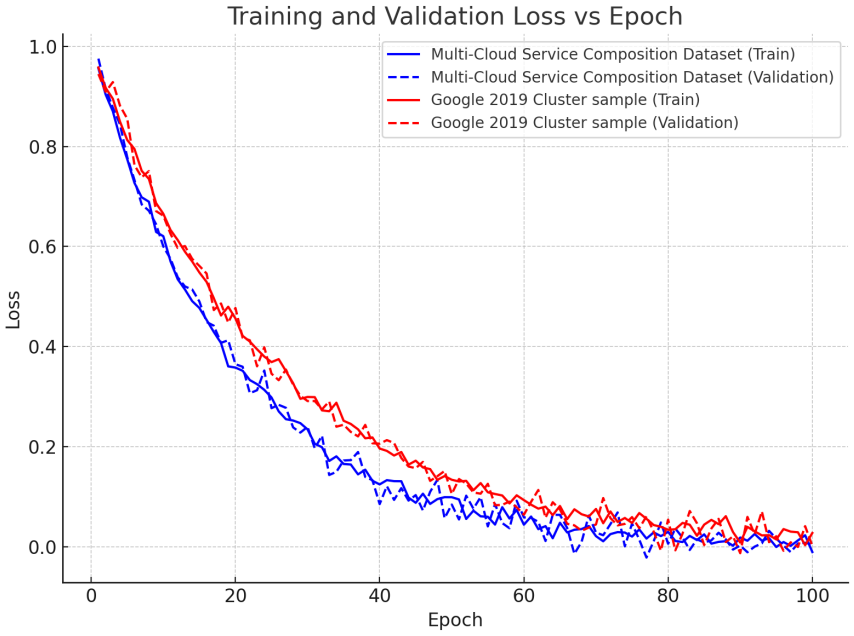

Figure 4. Comparison of Training and Validation Losses across Epochs (Multi-Cloud Service Composition Dataset and Google 2019 Cluster Sample)

The figure 4 clearly shows the dynamic changes in training and validation losses across epochs for the Multi-Cloud Service

ARTICLE IN PRESS

Composition Dataset and the Google 2019 Cluster sample. Overall, both training and validation losses exhibit a significant decreasing trend as the number of epochs increases. In the early stages of training, the training loss (blue line) and validation loss (blue dashed line) on the Multi-Cloud dataset differ from the training loss (red line) and validation loss (red dashed line) on the Google dataset. This disparity may be due to differences in data distribution, scenario complexity, and other factors between the two datasets, leading to different initial learning results for the model. However, as training progresses, the four curves gradually converge and stabilize at lower loss levels. By the later stages of training, the training and validation losses for the Multi-Cloud dataset closely align with those for the Google dataset, demonstrating that the model can effectively capture the core patterns in the data from both datasets. Furthermore, the change trends of training and validation losses are highly consistent, with both ultimately maintaining low loss values. This indicates that the model architecture and training strategy are well-suited, allowing the model to learn efficiently from the training set and demonstrate excellent generalization ability on the validation set. There is no significant overfitting (where validation loss is much higher than training loss) or underfitting (where the loss does not effectively decrease), providing strong evidence that the model can stably and efficiently learn and optimize in different scenarios, such as multi-cloud service composition and dynamic clusters, laying a solid foundation for its performance in real-world applications.

Table 6. Ablation Study Results: Impact of TSL-HRL Module Ablation on Prediction Accuracy

Model Configuration	**Multi-Cloud Dataset**		**Google 2019 Cluster Sample**	
	MAE	RMSE	MAE	RMSE
TSL-HRL (Full Model)	0.06±0.005*	0.08±0.007*	0.07±0.006*	0.09±0.008*
Without TSMixer Module	0.09±0.008	0.12±0.010	0.10±0.009	0.13±0.011
Without LDS Module	0.08±0.007	0.11±0.009	0.09±0.008	0.12±0.010
Without Hierarchical RL Module	0.11±0.010	0.14±0.012	0.12±0.011	0.15±0.013
Basic RL Framework (No Special Modules)	0.15±0.012	0.18±0.015	0.16±0.013	0.20±0.016

* Statistically significant improvement (two-tailed t-test, $p < 0.05$) over all ablated configurations. Key p-values: TSL-HRL vs Without Hierarchical RL (MAE: 0.027, RMSE: 0.031); TSL-HRL vs Basic RL Framework (MAE: 0.019, RMSE: 0.023). All p-values are calculated from 5 independent runs.

ARTICLE IN PRESS

The ablation study results in Table 6 show that the collaborative effect of the core modules in TSL-HRL is significant in improving prediction accuracy. When the TSMixer module is removed, the MAE for Multi-Cloud increases to 0.09 with a 50% increase and RMSE increases to 0.12 with a 50% increase. For the Google dataset, the MAE and RMSE rise to 0.10 and 0.13 respectively. This highlights TSMixer's key role in capturing deep temporal dependencies, as its absence leaves the model unable to recognize periodic resource usage patterns. When the LDS module is removed, the MAE and RMSE for Multi-Cloud increase to 0.08 and 0.11 with respective increases of 33.3% and 37.5%. Similar significant increases in errors are observed for the Google dataset. This confirms the module's value in dynamic state calibration, as losing it makes the model more sensitive to sudden burst traffic and node state changes. The removal of the Hierarchical RL module has an even more significant impact on accuracy. The MAE for Multi-Cloud spikes to 0.11 with an 83.3% increase from the full model, and RMSE reaches 0.14. For the Google dataset, the MAE and RMSE are 0.12 and 0.15 respectively. This underscores the importance of hierarchical decision-making, as the lack of global planning and local optimization coordination leads to suboptimal scheduling choices. When only the basic RL framework is retained, the errors for both datasets are at their highest. The MAE values are 0.15 and 0.16 while the RMSE values are 0.18 and 0.20. This further indicates that TSL-HRL, through its three-level architecture design of time-series feature modeling dynamic state perception and hierarchical strategy optimization, precisely models resource management problems across multiple scenarios. The synergy of these modules addresses key challenges in multi-cloud scheduling, making the integrated design critical for superior performance.

ARTICLE IN PRESS

Discussion

Within the research framework of multi-cloud resource scheduling and cluster dynamic management, TSL-HRL demonstrates significant advantages in resource utilization efficiency and dynamic adaptability through its hierarchical reinforcement learning architecture and multi-module collaboration mechanism. Quantitative experimental results further validate these advantages: On the Multi-Cloud Service Composition Dataset, TSL-HRL reduces inference time by 39.6% versus Transformer+PPO and improves resource utilization by 18.3% versus the Zhan model; on the Google 2019 Cluster sample, it shortens training time by 45.5% versus the Zi model and cuts peak GPU memory usage by 38.7% versus ALMAZROI. Its core value lies in the construction of a systematic solution based on time-series feature modeling - dynamic state perception - hierarchical strategy optimization, effectively addressing the heterogeneity and high dynamism challenges in multi-cloud environments. However, the current research still has expandable technical boundaries and applicable scenarios to explore.

From the model design perspective, although TSL-HRL has been validated for effectiveness on two typical datasets, its horizontal scalability in ultra-large-scale multi-cloud clusters—such as those with node scales exceeding one hundred thousand—has not been fully verified. Specifically, in cross-cloud service provider resource scheduling, balancing the compatibility of different vendors' API interfaces with the consistency of strategy execution remains an engineering challenge to be addressed. Additionally, the model's current approach to modeling complex constraints such as network transmission latency and data privacy compliance is still based on a single dimension. It has not yet formed a unified framework for multi-objective optimization, which may lead to conflicts in resource allocation and business QoS or Quality of Service during actual deployment.

In terms of application scenario expansion, the research does not cover edge-cloud collaborative scheduling in multi-cloud environments. With the widespread adoption of cloud-edge-end integrated architectures, edge nodes' low-latency computing complements cloud resources' high throughput, yet edge devices' limited computing power and intermittent network connections impose new requirements for lightweight and fault-tolerant design of the existing hierarchical decision-making mechanism. Notably, this gap hinders adaptation to sustainable 6G's distributed resource coordination demands and digital twin systems' real-time edge-cloud interaction needs. Meanwhile, green computing-oriented energy optimization has not been integrated into the model design: current experiments focus solely on resource utilization and scheduling latency, without considering cloud providers' carbon footprint differences or resource-related energy costs. This limits practical applicability amid global carbon neutrality goals, as well as alignment with 6G and digital twin systems' sustainability requirements.

Future research can be conducted in three areas: First, for the edge-cloud collaborative scenario, design a lightweight hierarchical strategy module by lightweighting the TSMixer module to adapt to the limited computing power of edge devices and using federated learning technology to implement dynamic collaboration between localized decision-making at the edge nodes and global strategies at the cloud, improving the system's robustness in distributed heterogeneous environments. Second, construct a multi-objective optimization framework that adopts an adaptive multi-objective swarm intelligence mechanism validated in existing multi-cloud optimization studies, incorporates energy consumption costs, data transmission security, and other metrics into the reward function of the Hierarchical RL, forming a collaborative optimization mechanism for resource scheduling, service quality, and green computing. Finally, explore the application of self-supervised learning and meta-learning techniques in model initialization, reducing dependence on large-scale labeled data and enhancing the rapid adaptation ability of TSL-HRL to emerging multi-cloud service scenarios. Through deepening research in these directions, it is expected to promote multi-cloud resource management technology from a single efficiency optimization stage to an efficient, green, and ubiquitous intelligent phase.

Data Availability

The datasets used in this study are publicly available. The Google 2019 Cluster Sample dataset is available from https://www.kaggle.com/datasets/derrickmwiti/google-2019-cluster-sample. The Multi-Cloud Service Composition dataset is available from

ARTICLE IN PRESS

https://www.kaggle.com/datasets/ziya07/multi-cloud-service-composition-dataset.

Author Contributions

Hong Liu, Shulei Zhang, and Li Li were responsible for data collection and analysis, and contributed to the writing of the manuscript. Tingting Sun and Wei Xue conducted the experiments and contributed to the interpretation of the results. Xiaoxia Yao and Yong Xu provided critical revisions to the manuscript. All authors reviewed and approved the final manuscript.

Competing interests:

The authors declare that they have no competing interests. The research was conducted independently, with no financial or non-financial interests influencing the results.

Funding:

The Authors received NO Funding for this work.

References

1. Tang, X. *et al.* Computing power network: The architecture of convergence of computing and networking towards 6g requirement. *China communications* **18**, 175–185 (2021).
2. Yukun, S. *et al.* Computing power network: A survey. *China Commun.* **21**, 109–145 (2024).
3. Lei, B., Zhao, Q. & Mei, J. Computing power network: An interworking architecture of computing and network based on ip extension. In *2021 IEEE 22nd international conference on high performance switching and routing (HPSR)*, 1–6 (IEEE, 2021).
4. Chen, Y. *et al.* Research progress on computing power network for the power industry. In *2024 IEEE 12th International Conference on Information, Communication and Networks (ICICN)*, 201–210 (IEEE, 2024).
5. Tang, Q. *et al.* Stochastic computation offloading for leo satellite edge computing networks: A learning-based approach. *IEEE Internet Things J.* **11**, 5638–5652 (2023).
6. Zhong, Y. *et al.* A memristor-based analogue reservoir computing system for real-time and power-efficient signal processing. *Nat. Electron.* **5**, 672–681 (2022).
7. Jia, M., Zhang, L., Wu, J., Guo, Q. & Gu, X. Joint computing and communication resource allocation for edge computing towards huge leo networks. *China Commun.* **19**, 73–84 (2022).
8. Zi, Y. Time-series load prediction for cloud resource allocation using recurrent neural networks. *J. Comput. Technol. Softw.* **3** (2024).
9. Daraghmeh, M., Agarwal, A., Manzano, R. & Zaman, M. Time series forecasting using facebook prophet for cloud resource management. In *2021 IEEE International Conference on Communications Workshops (ICC Workshops)*, 1–6 (IEEE, 2021).
10. Ouhame, S., Hadi, Y. & Ullah, A. An efficient forecasting approach for resource utilization in cloud data center using cnn-lstm model. *Neural Comput. Appl.* **33**, 10043–10055 (2021).
11. Ning, E., Wang, Y., Wang, C., Zhang, H. & Ning, X. Enhancement, integration, expansion: Activating representation of detailed features for occluded person re-identification. *Neural Networks* **169**, 532–541 (2024).
12. Al-Jumaili, A. H. A., Muniyandi, R. C., Hasan, M. K., Paw, J. K. S. & Singh, M. J. Big data analytics using cloud computing based frameworks for power management systems: Status, constraints, and future recommendations. *Sensors* **23**, 2952 (2023).

ARTICLE IN PRESS

ARTICLE IN PRESS

13. Zhu, J. *et al.* Variational mode decomposition and sample entropy optimization based transformer framework for cloud resource load prediction. *Knowledge-Based Syst.* **280**, 111042 (2023).

14. Lv, L. *et al.* A vmd and lstm based hybrid model of load forecasting for power grid security. *IEEE Transactions on Ind. Informatics* **18**, 6474–6482 (2021).

15. Hameed, A. *et al.* Toward qos prediction based on temporal transformers for iot applications. *IEEE Transactions on Netw. Serv. Manag.* **19**, 4010–4027 (2022).

16. Chona, E., Denkovski, D. & Rakovic, V. Transformer-driven dynamic resource reconfiguration in cellular networks. In *IEEE EUROCON 2025-21st International Conference on Smart Technologies*, 1–6 (IEEE, 2025).

17. Hu, Y., Xie, X., Lei, T., Shi, R. & Wong, M. A sliding-kernel computation-in-memory architecture for convolutional neural network. *Adv. Sci.* **11**, 2407440 (2024).

18. Lu, K., Huo, M., Li, Y., Zhu, Q. & Chen, Z. Ct-patchtst: Channel-time patch time-series transformer for long-term renewable energy forecasting. *arXiv preprint arXiv:2501.08620* (2025).

19. Devi, D. & Winster, S. G. Azo-fmrgaformer: a hybrid model for cloud resource allocation using transformer-based time-series forecasting. *Signal, Image Video Process.* **19**, 732 (2025).

20. Nazir, A., Shaikh, A. K., Shah, A. S. & Khalil, A. Forecasting energy consumption demand of customers in smart grid using temporal fusion transformer (tft). *Results Eng.* **17**, 100888 (2023).

21. Rathi, R., Narkhede, N. & Hasamnis, M. Design and implementation of first-in-first-out (fifo) buffer for distributed load balancing systems. In *International Conference on Smart Computing and Communication*, 11–21 (Springer, 2024).

22. Jenila, C. *et al.* Efficient synchronous fifo design: A structured approach using asic methodology. In *2025 7th International Conference on Inventive Material Science and Applications (ICIMA)*, 93–100 (IEEE, 2025).

23. Cosic, A., Stadler, M., Mansoor, M. & Zellinger, M. Mixed-integer linear programming based optimization strategies for renewable energy communities. *Energy* **237**, 121559 (2021).

24. Alhasnawi, B. N., Jasim, B. H., Sedhom, B. E. & Guerrero, J. M. A new communication platform for smart ems using a mixed-integer-linear-programming. *Energy Syst.* **16**, 471–488 (2025).

25. Cao, W. *et al.* Flexible computing: A new framework for improving resource allocation and scheduling in elastic computing. *IEEE Transactions on Serv. Comput.* (2024).

26. Nguyen, C. T. *et al.* Elastic resource allocation for coded distributed computing over heterogeneous wireless edge networks. *IEEE Transactions on Wirel. Commun.* **22**, 2636–2649 (2022).

27. Ahmadi, M., Hadi, M. & Pakravan, M. R. Power-efficient joint dynamic resource allocation in virtualized inter-data center elastic optical networks. *IEEE Access* **12**, 75599–75609 (2024).

28. Kamble, T. *et al.* Predictive resource allocation strategies for cloud computing environments using machine learning. *J. Electr. Syst.* **19** (2023).

29. Easwaran, S. & Shadaram, M. Enhanced resource allocation in elastic optical network using deep learning and optimization process. *Opt. Fiber Technol.* **93**, 104210 (2025).

30. Chen, Q. *et al.* A survey on resource management in joint communication and computing-embedded sagin. *IEEE Commun. Surv. & Tutorials* **27**, 1911–1954 (2024).

31. Bai, W., Zhu, J., Huang, S. & Zhang, H. A queue waiting cost-aware control model for large scale heterogeneous cloud datacenter. *IEEE Transactions on Cloud Comput.* **10**, 849–862 (2022).

32. Lee, J., Solat, F., Kim, T. Y. & Poor, H. V. Federated learning-empowered mobile network management for 5g and beyond networks: From access to core. *IEEE Commun. Surv. & Tutorials* **26**, 2176–2212 (2024).

ARTICLE IN PRESS

ARTICLE IN PRESS

33. Cheng, N. *et al.* Toward enhanced reinforcement learning-based resource management via digital twin: Opportunities, applications, and challenges. *IEEE Netw.* **39**, 189–196 (2024).

34. Ning, E., Wang, C., Zhang, H., Ning, X. & Tiwari, P. Occluded person re-identification with deep learning: A survey and perspectives. *Expert. Syst. with Appl.* **239**, 122419, DOI: https://doi.org/10.1016/j.eswa.2023.122419 (2024).

35. Hasan, M. K. *et al.* Federated learning for computational offloading and resource management of vehicular edge computing in 6g-v2x network. *IEEE Transactions on Consumer Electron.* **70**, 3827–3847 (2024).

36. Nandhakumar, A. R., Baranwal, A., Choudhary, P., Golec, M. & Gill, S. S. Edgeaisim: A toolkit for simulation and modelling of ai models in edge computing environments. *Meas. Sensors* **31**, 100939 (2024).

37. Tirlangi, S. *et al.* Cloud computing and machine learning in the green power sector: Data management and analysis for sustainable energy. In *Developments Towards Next Generation Intelligent Systems for Sustainable Development*, 148–179 (IGI Global Scientific Publishing, 2024).

38. Kristian, A., Goh, T. S., Ramadan, A., Erica, A. & Sihotang, S. V. Application of ai in optimizing energy and resource management: Effectiveness of deep learning models. *Int. Transactions on Artif. Intell.* **2**, 99–105 (2024).

39. Samha, A. K. Strategies for efficient resource management in federated cloud environments supporting infrastructure as a service (iaas). *J. Eng. Res.* **12**, 101–114 (2024).

40. Zhu, J. *et al.* Adaptive multi-objective swarm intelligence for containerized microservice deployment. *Futur. Gener. Comput. Syst.* **174**, 108012 (2026).

41. Huang, F. *et al.* Drmq: Dynamic resource management for enhanced qos in collaborative edge-edge industrial environments. *IEEE Transactions on Serv. Comput.* (2025).

42. Zhan, J. Elastic scheduling of micro-modules in edge computing based on lstm prediction. *J. Comput. Technol. Softw.* **4** (2025).

43. Almazroi, A. A. & Ayub, N. Multi-task learning for electricity price forecasting and resource management in cloud based industrial iot systems. *IEEE Access* **11**, 54280–54295 (2023).

44. Aldossary, M., Alharbi, H. A. & Ayub, N. Exploring multi-task learning for forecasting energy-cost resource allocation in iot-cloud systems. *Comput. Mater. & Continua* **79** (2024).

45. Lan, S., Song, S., Wang, Z. & Long, Y. Multi-objective optimization for cost and latency in computing force network. In *Proceedings of the 2nd International Seminar on Artificial Intelligence, Networking and Information Technology (ANIT 2023)*, 291–300, DOI: 10.5220/0012281700003807 (SCITEPRESS – Science and Technology Publications, Lda., 2023). Paper published under CC license (CC BY-NC-ND 4.0).

46. Anslam Sibi, S. & Sherly Puspha Annabel, L. Optimizing energy resources in wsns: Arima feature selection meets adaptive reinforcement learning. *Fiber Integr. Opt.* **43**, 69–96 (2024).

47. Zhang, S. & Cao, R. Multi-objective optimization for uav-enabled wireless powered iot networks: An lstm-based deep reinforcement learning approach. *IEEE Commun. Lett.* **26**, 3019–3023 (2022).

48. Mehroliya, S., Tomar, S., Arya, A. & Verma, A. A novel hybrid ga-pso algorithm-based optimization of transmission and expansion planning. *SN Comput. Sci.* **4**, 690 (2023).

49. Gholipour, N., de Assuncao, M. D., Agarwal, P., Gascon-Samson, J. & Buyya, R. Tpto: A transformer-ppo based task offloading solution for edge computing environments. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*, 1115–1122 (IEEE, 2023).

ARTICLE IN PRESS